

Golden Dataset & Coder Rules

How to teach the AI from coder feedback

API reference for the Valerion CPU pipeline:

review corrections, coder rules, Qdrant write endpoints, and the search endpoints the agents call on the next document.

Audience: backend engineers, coding leads, admins

Generated: 2026-05-10

1. What this guide covers

Valerion CPU does not have a single endpoint literally named “**golden dataset**”. Instead, the “golden dataset” is built from two streams of human input that are both written into Qdrant and pulled back by the agents on every future report:

- **Coder corrections** — submitted from the review screen (*EDIT / DELETE / ADD* on AI-suggested codes). Stored in the **correction_memory** Qdrant collection.
- **Coder rules** — standing instructions added by an admin via “*Add coder rule*”. Stored in the **human_coder_rules** Qdrant collection.

On the next document, the orchestrator pulls both back from Qdrant and injects them into agent prompts, so the AI follows the rules and avoids repeating mistakes.

Services involved

Gateway	Public entry, auth, proxies to orchestrator
Orchestrator	Owns Postgres tables (source of truth) and exposes the user-facing rule and review APIs
RAG service	Owns Qdrant; exposes internal /write and /search endpoints called by orchestrator and agents

Qdrant collections written by these flows

Collection	Stored content	Written by	Read by
correction_memory	Coder corrections (EDIT / DELETE / ADD actions).	POST /write/correction_memory (RAG); original corrected codes, re	POST /ingest/correction_memory (RAG); original corrected codes, re
human_coder_rules	Standing coder rules. Embedding = rule text; payload = rule text applies to (PQS), /write/coder_rules	POST /write/coder_rules (RAG); /write/coder_rules	POST /ingest/coder_rules (RAG); /search/coder_rules

Note: code-catalog collections (icd10_cm_codes, icd10_pcs_codes, cpt, coding_guidelines_*, mimic_*) are read-only at runtime and loaded out of band — they are not part of this guide.

2. “Add coder rule” APIs

A coder rule is a short instruction the AI must follow on every future report it processes (e.g. “Always sequence sepsis before the underlying infection unless documentation states otherwise”). Rules live in Postgres (**coder_rules**) and are mirrored into Qdrant (**human_coder_rules**) so agents can retrieve them by similarity.

Priority semantics

- **HIGH** priority rules — always injected into agent prompts, regardless of similarity.
- **NORMAL** priority rules — pulled by similarity search against the de-identified clinical text of the current report (top-K).
- **active=False** rules are excluded from all retrieval immediately, no restart required.

2.1 POST /api/rules — create a rule

Service	Orchestrator (also reachable through gateway)
Source	orchestrator/app/routers/rules.py:105
Auth	JWT required at the gateway. Production should restrict to ADMIN role.
Status	201 Created
Side effects	1) INSERT into Postgres coder_rules. 2) Calls RAG /write/coder_rule to upsert into Qdrant human_coder_rules

Request body

```
{
  "rule_text": "Always assign Z51.11 when the encounter is for chemotherapy.",
  "applies_to": "ICD-CM",           // ICD-CM | ICD-PCS | CPT | ALL
  "priority":   "HIGH",             // HIGH | NORMAL
  "created_by": "5e4c...-uuid"      // user UUID
}
```

Response

```
{
  "id":           "f1b2...-uuid",
  "rule_text":    "Always assign Z51.11 when ...",
  "applies_to":   "ICD-CM",
  "priority":     "HIGH",
  "active":       true,
  "created_by":   "5e4c...-uuid",
  "created_at":   "2026-05-10T10:24:31Z",
  "qdrant_synced": true
}
```

Example

```
curl -X POST https://api.example.com/api/rules \
-H "Authorization: Bearer $JWT" \
-H "Content-Type: application/json" \
-d '{
  "rule_text": "Always assign Z51.11 when the encounter is for chemotherapy.",
  "applies_to": "ICD-CM",
  "priority":   "HIGH",
  "created_by": "5e4c0c84-2a7c-4ad2-9b2a-7d9c3a5e9c11"
}'
```

Tip: rule_text must be at least 10 characters. Write the rule as a self-contained imperative — the agent sees just the text, not the title.

2.2 GET /api/rules — list rules

Service	Orchestrator
Source	orchestrator/app/routers/rules.py:173
Reads from	Postgres (Qdrant is the search index, not the source of truth)
Query params	priority (HIGH NORMAL), applies_to (ICD-CM ICD-PCS CPT ALL), include_inactive (bool, default false)

Response

```
{
  "rules":      [ { ...rule object... }, ... ],
  "total":      12,
  "high_count": 3,
  "normal_count": 9
}
```

Example

```
curl "https://api.example.com/api/rules?priority=HIGH&applies_to=ICD-CM" \
-H "Authorization: Bearer $JWT"
```

2.3 GET /api/rules/{rule_id} — fetch one rule

Service	Orchestrator
Source	orchestrator/app/routers/rules.py:217
404	If rule_id is unknown

2.4 PATCH /api/rules/{rule_id}/deactivate

Service	Orchestrator
Source	orchestrator/app/routers/rules.py:229
Auth	Production should restrict to ADMIN role
Side effects	1) UPDATE coder_rules SET active=false. 2) Calls RAG /write/coder_rule/{id}/deactivate. Rule is excluded from search.

Response

```
{
  "rule_id":      "f1b2...-uuid",
  "active":       false,
  "qdrant_synced": true,
  "message":      "Rule deactivated"
}
```

Example

```
curl -X PATCH https://api.example.com/api/rules/f1b2.../deactivate \
-H "Authorization: Bearer $JWT"
```

There is no DELETE — rules are soft-deactivated so the audit trail stays intact in Postgres.

3. “Golden dataset” APIs (review & corrections)

The golden dataset is built from coder review actions. Whenever a coder **EDITS**, **DELETES**, or **ADDs** a code on the review screen, the orchestrator writes a row to the **coder_corrections** table and mirrors it into the **correction_memory** Qdrant collection. ACCEPT actions are recorded for audit but do *not* write to Qdrant — there is nothing to learn from a confirmed-correct prediction.

3.1 POST /api/review/report/{report_id}/submit

Service	Orchestrator
Source	orchestrator/app/routers/review.py:535
Auth	JWT required (gateway)
Side effects	Per action: writes ReviewedCode row (audit). For EDIT/DELETE/ADD: also writes a CoderCorrection row

Request body

```
{
  "coder_id": "5e4c...-uuid",
  "actions": [
    {
      "action": "ACCEPT",
      "predicted_code_id": "p1-uuid"
    },
    {
      "action": "EDIT",
      "predicted_code_id": "p2-uuid",
      "correct_code": "I50.32",
      "correct_description": "Chronic diastolic (congestive) heart failure",
      "reason": "Documentation specifies chronic diastolic HF, not unspecified."
    },
    {
      "action": "DELETE",
      "predicted_code_id": "p3-uuid",
      "reason": "Code not supported by documentation; ruled out on page 4."
    },
    {
      "action": "ADD",
      "correct_code": "E11.9",
      "correct_description": "Type 2 diabetes mellitus without complications",
      "code_type": "secondary",
      "sequence_pos": 3,
      "reason": "Documented in HPI but missed by AI."
    }
  ]
}
```

Field rules

- **action** — one of ACCEPT, EDIT, DELETE, ADD.
- **predicted_code_id** — required for ACCEPT/EDIT/DELETE; omit for ADD.
- **correct_code** — required for EDIT and ADD.
- **code_type** — required for ADD: *primary* | *secondary* | *procedure* | *cpt*.

- **reason** — required for EDIT/DELETE/ADD; minimum 20 characters. This text is what trains the AI, so be specific and clinical.

Response

```
{
  "report_id":      "r1-uuid",
  "total_actions":  4,
  "accepted":       1,
  "edited":         1,
  "deleted":        1,
  "added":          1,
  "corrections_written": 3,
  "qdrant_sync_failures": 0,
  "results": [
    { "action": "ACCEPT", "ok": true, "code": "...", },
    { "action": "EDIT",   "ok": true, "correction_id": "...", "qdrant_synced": true },
    ...
  ]
}
```

3.2 POST /api/review/encounter/{encounter_id}/submit

Service	Orchestrator
Source	orchestrator/app/routers/review.py:603
Body	Same shape as 3.1; predicted_code_id values come from the encounter-level review payload
Side effects	Same: ReviewedCode rows + CoderCorrection rows + Qdrant correction_memory upserts

Use the encounter endpoint when a coder reviews a multi-report encounter as one unit; use the report endpoint for single-report review.

3.3 What gets written to Qdrant

Collection	correction_memory
Embed text	Composed from wrong code, correct code, descriptions, and reason — used for similarity match against
Payload	original_code, corrected_code, action_type (EDIT/DELETE/ADD), code_type, reason, coder_id, report
Point ID	Matches Postgres coder_corrections.id for traceability

Two-phase sync: Postgres is always the source of truth. The Qdrant write is best-effort — failures are logged and reported in *qdrant_sync_failures* on the response, but the correction itself is still safely persisted.

4. RAG service /write endpoints (internal)

These endpoints sit on the RAG service and are normally called by the orchestrator, not directly by clients. They are documented here because they define exactly what gets written into Qdrant — useful for debugging, backfills, or rebuilding the index from Postgres.

4.1 POST /write/correction_memory

Service	RAG service (port 8200)
Source	rag_service/app/routers/write.py:67
Caller	orchestrator → rag_writer.write_correction_memory()
Collection	correction_memory

Request body

```
{
  "embed_text": "chronic diastolic heart failure -> I50.32 ... reason: ...",
  "payload": {
    "original_code": "I50.9",
    "corrected_code": "I50.32",
    "action_type": "EDIT",
    "code_type": "primary",
    "reason": "Documentation specifies chronic diastolic HF.",
    "coder_id": "5e4c...-uuid",
    "report_id": "r1-uuid",
    "reviewed_at": "2026-05-10T10:24:31Z"
  },
  "point_id": "optional-uuid-defaults-to-postgres-correction-id"
}
```

Response

```
{ "success": true, "point_id": "...", "collection": "correction_memory" }
```

4.2 POST /write/coder_rule

Service	RAG service
Source	rag_service/app/routers/write.py:127
Caller	orchestrator → rag_writer.write_coder_rule()
Collection	human_coder_rules

Request body

```
{
  "rule_id": "f1b2...-uuid", // Postgres rule.id, used as Qdrant point id
  "rule_text": "Always assign Z51.11 when the encounter is for chemotherapy.",
  "applies_to": "ICD-CM",
  "priority": "HIGH",
  "active": true,
  "created_by": "5e4c...-uuid"
}
```

Response

```
{ "success": true, "point_id": "...", "collection": "human_coder_rules" }
```

4.3 PATCH /write/coder_rule/{rule_id}/deactivate

Service	RAG service
Source	rag_service/app/routers/write.py:182
Caller	orchestrator → rag_writer.deactivate_coder_rule()
Effect	Sets active=False on the Qdrant point payload — rule is excluded from search immediately

```
{ "success": true, "point_id": "f1b2...-uuid" }
```

5. How the AI uses this data next time

On every new report, the orchestrator calls these search endpoints on the RAG service and injects the results into the agent prompts. This is what makes corrections and rules *actually change* the AI's behavior on subsequent documents.

5.1 POST /search/coder_rules

Service	RAG service
Source	rag_service/app/routers/search.py:104
Reads	human_coder_rules collection

Request body

```
{
  "clinical_text": "<de-identified clinical text from current report>",
  "top_k_normal": 5
}
```

Response

```
{
  "high_priority_rules": [ { id, rule_text, applies_to, score, ... }, ... ],
  "normal_priority_rules": [ { id, rule_text, applies_to, score, ... }, ... ],
  "total": 8
}
```

Search logic

- **HIGH**: payload filter *priority=HIGH AND active=True*. No similarity threshold; all returned and injected verbatim.
- **NORMAL**: vector similarity over *clinical_text*, filtered by *priority=NORMAL AND active=True*, limited to *top_k_normal*.

5.2 POST /search/correction_memory

Service	RAG service
Source	rag_service/app/routers/search.py:90
Reads	correction_memory collection

Request body

```
{
  "clinical_text": "<de-identified clinical text from current report>",
  "top_k": 5,
  "threshold": 0.82
}
```

Response

```
{
  "corrections": [ { id, original_code, corrected_code, reason, action_type, score, ... }, ... ],
  "count": 3
}
```

```
}
```

Only corrections with similarity score above *threshold* are returned, so unrelated past cases don't pollute the prompt.

6. End-to-end flows

6.1 Adding a coder rule

1. Admin → POST /api/rules (gateway → orchestrator)
2. Orchestrator INSERT INTO coder_rules (Postgres = source of truth)
3. Orchestrator → POST /write/coder_rule (RAG service)
4. RAG embeds rule_text, upserts into Qdrant
collection=human_coder_rules, point_id = rule.id
5. Response → qdrant_synced: true
6. Next report processed: orchestrator calls
POST /search/coder_rules → HIGH always injected,
NORMAL injected if similar to clinical_text.

6.2 Submitting review corrections

1. Coder → POST /api/review/report/{id}/submit (gateway → orchestrator)
2. For each action:
ACCEPT → INSERT reviewed_codes (audit only)
EDIT/DELETE/ADD → INSERT reviewed_codes
INSERT coder_corrections
POST /write/correction_memory (RAG → Qdrant)
3. RAG embeds (codes + reason + clinical context),
upserts into correction_memory, point_id = coder_corrections.id
4. Response → corrections_written, qdrant_sync_failures
5. Next report processed: orchestrator calls
POST /search/correction_memory → top-K similar past corrections
injected into the relevant agent prompt.

6.3 Deactivating a rule

1. Admin → PATCH /api/rules/{id}/deactivate
2. Orchestrator UPDATE coder_rules SET active=false
3. Orchestrator → PATCH /write/coder_rule/{id}/deactivate
4. RAG sets payload active=false on the Qdrant point
5. Rule is excluded from /search/coder_rules immediately – no service restart.

7. Auth, roles, and operational notes

- All **/api/...** endpoints sit behind the gateway and require a JWT.
- Rule mutation endpoints (POST **/api/rules**, PATCH **/api/rules/{id}/deactivate**) should be restricted to **ADMIN** role in production. The route handlers carry a TODO/comment to that effect.
- Review-submit endpoints require an authenticated coder; the *coder_id* in the body must match the JWT subject in production deployments.
- RAG **/write/...** and **/search/...** endpoints are internal — they are not exposed through the gateway and should only be reachable on the private docker/k8s network.
- Postgres is always source of truth. If Qdrant is wiped, you can rebuild **human_coder_rules** by replaying every active row in *coder_rules* through POST **/write/coder_rule**, and rebuild **correction_memory** by replaying *coder_corrections* through POST **/write/correction_memory**.
- Qdrant payloads contain **no PHI and no clinical narrative**. Only structured fields and de-identified *embed_text* are stored. PHI redaction happens upstream in the OCR / de-identification pipeline.

Common pitfalls

- **Reason too short** — review actions with reason < 20 chars are rejected. The reason field is what makes a correction useful for the AI later.
- **Forgetting code_type on ADD** — required: primary | secondary | procedure | cpt.
- **Treating ACCEPT as training data** — it isn't. ACCEPT is audit-only. Only EDIT/DELETE/ADD reach Qdrant.
- **Expecting a hard delete** — there is no DELETE on rules. Use PATCH **/api/rules/{id}/deactivate**.
- **qdrant_sync_failures > 0** — Postgres write succeeded but RAG/Qdrant did not. Rule/correction is safe; re-run the sync from Postgres → RAG to backfill.

— End of guide —